Desafio progressivo de Python

5 questões práticas e analíticas | Listas, repetições e entrada de dados

Nome: _____ Turma: _____ Data: ____/____/_____

Orientações: resolva as questões na ordem apresentada. Entregue o código de cada programa e as justificativas solicitadas. Utilize Python 3. Não é necessário criar interface gráfica. Considere entradas válidas nas questões 1 a 4; na questão 5, valide os dados conforme solicitado.

1. Leitura e classificação de valores | Nível inicial

Crie um programa que leia 6 números inteiros e os armazene em uma lista inicialmente vazia, utilizando `append()`.

a) Use `for` na leitura e mostre a lista completa ao final.
b) Percorra a lista e conte quantos valores são positivos, negativos e iguais a zero.
c) Exiba cada elemento acompanhado de seu índice, começando em zero.
d) Explique por que tentar atribuir diretamente `numeros[0] = 10` a uma lista vazia não substitui o uso de `append()`.

2. Estatísticas sem funções prontas | Nível básico

Considere a lista de temperaturas abaixo. Seu programa deve continuar funcionando se os valores ou a quantidade de elementos forem alterados.

```
temperaturas = [28, 31, 26, 33, 29, 25, 32]
```

a) Calcule a soma e a média sem utilizar `sum()`.
b) Descubra o maior e o menor valor sem utilizar `max()`, `min()` ou ordenação.
c) Gere uma nova lista com as temperaturas estritamente acima da média.
d) Caso a lista esteja vazia, exiba uma mensagem e não realize os cálculos.
e) Justifique por que iniciar o maior e o menor com o primeiro elemento é mais adequado do que iniciá-los sempre com zero.

3. União e interseção sem repetições | Nível intermediário

Duas turmas escolheram linguagens de programação para estudar:

```
A = ["Python", "Java", "C", "Python", "JavaScript"]
B = ["Java", "Go", "Python", "Rust", "Go"]
```

a) Crie uma lista com todas as linguagens, sem repetições: primeiro as de A e depois as novidades de B.
b) Crie outra lista apenas com as linguagens presentes nas duas turmas, sem repetições e na ordem da primeira ocorrência em A.
c) Crie uma terceira lista com as linguagens presentes em apenas uma das turmas, sem repetições: primeiro as exclusivas de A e depois as exclusivas de B.
d) Use laços e testes de pertinência (`in` / `not in`), sem utilizar `set()`. Mostre as três listas e a quantidade de elementos de cada uma.

Desafio progressivo de Python

Continuação | Busca, atualização e integração de conceitos

4. Busca com múltiplas ocorrências e substituição | Nível avançado

Considere uma lista de códigos de produtos. Um mesmo código pode aparecer mais de uma vez:

```
codigos = [12, 7, 12, 9, 7, 15, 12]
```

a) Solicite um código ao usuário e percorra toda a lista para armazenar, em outra lista, todos os índices em que ele aparece.
b) Informe os índices encontrados e o total de ocorrências. Se não houver ocorrência, mostre uma mensagem específica.
c) Se o código existir, pergunte se o usuário deseja substituir todas as ocorrências por outro código inteiro. Faça a alteração somente se a resposta for `s` ou `S`.
d) Mostre a lista original e a lista após a operação. Preserve a lista original por meio de uma cópia.
e) Não use `index()` nem `count()`. Explique por que um `break` logo após a primeira ocorrência impediria atender ao item (a).

Teste obrigatório: ao procurar 12 na lista fornecida, os índices devem ser [0, 2, 6]. Teste também um código inexistente e uma substituição em que o novo código já esteja presente.

5. Cadastro interativo com relatório | Nível de integração

Construa um programa de cadastro de produtos usando duas listas paralelas: `nomes` e `quantidades`. O nome e a quantidade de um mesmo produto devem ocupar o mesmo índice nas duas listas. Não utilize dicionários.

a) Repita o cadastro até o usuário digitar `x` ou `X` no campo do nome. O marcador de saída não deve ser armazenado.
b) Remova espaços das extremidades do nome e rejeite nomes vazios. Considere nomes iguais independentemente de maiúsculas e minúsculas; por exemplo, "Mouse" e "mouse" representam o mesmo produto.
c) Para cada nome válido, solicite uma quantidade inteira maior que zero. Se a entrada for inválida, exiba uma mensagem e solicite a quantidade novamente, sem interromper o programa.
d) Se o produto já estiver cadastrado, some a quantidade informada à quantidade existente. Caso contrário, acrescente o produto às duas listas. Não conte tentativas inválidas como cadastros.
e) Ao encerrar, mostre cada produto e sua quantidade, o número de produtos distintos, a soma das quantidades e todos os produtos empatados com a maior quantidade. Se não houver produtos, informe isso sem tentar acessar uma posição da lista.
f) Conte quantos registros válidos de entrada foram realizados, incluindo as atualizações de produtos já existentes. Explique a diferença entre esse total e a quantidade de produtos distintos.

Restrições: use `while` para controlar a entrada e `for` em pelo menos uma etapa do relatório. Calcule a soma e a maior quantidade sem `sum()` e `max()`. Você pode utilizar métodos de strings e tratamento de exceções para validar as entradas.

Teste obrigatório: cadastre Mouse com 2 unidades, teclado com 3 e mouse com 1; depois encerre com X. O relatório deve indicar 2 produtos distintos, 6 unidades no total, 3 registros válidos e empate entre mouse e teclado, com 3 unidades cada.